

XÂY DỰNG MÔ PHỎNG HỆ THỐNG MINICLOUD CƠ BẢN

Mục tiêu

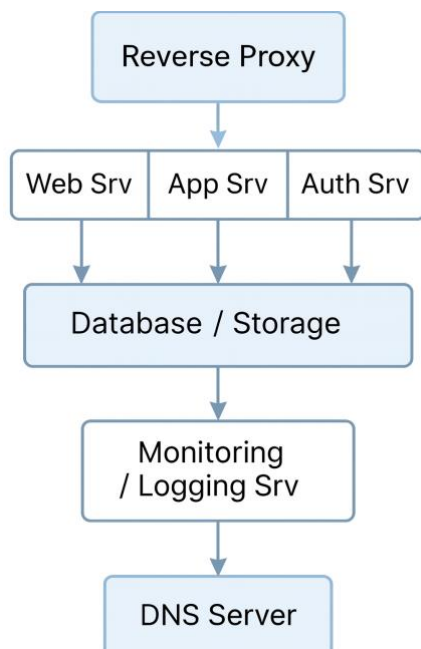
- Sinh viên xây dựng hệ thống Cloud thu nhỏ (**MyMiniCloud**) gồm 9 loại máy chủ cơ bản, mô phỏng các thành phần trong hạ tầng của một Cloud Platform (như AWS, Azure, GCP).
- Mỗi máy chủ chạy trong **container riêng biệt**, có thể giao tiếp qua **Docker Network**, và toàn hệ thống được triển khai & kiểm thử trên **máy local hoặc AWS EC2**.

9 LOẠI SERVER BẮT BUỘC CÓ TRONG HỆ THỐNG

STT	Loại Server	Vai trò & yêu cầu chính	Gợi ý công nghệ / image
1	Web Server	Cung cấp giao diện web tĩnh hoặc động (index, blog, dashboard)	nginx, apache, hoặc node:alpine
2	Application Server	Xử lý logic hoặc API, kết nối DB	python:flask, springboot, nodejs
3	Database Server	Lưu trữ dữ liệu người dùng hoặc nội dung web	mysql, mariadb, postgres
4	Authentication Server	Quản lý người dùng, token, đăng nhập	keycloak, authentik, hoặc custom Flask Auth
5	File Storage / Object Storage	Lưu trữ tệp, tài nguyên tĩnh	minio, nextcloud, hoặc owncloud
6	DNS / Name Service	Quản lý ánh xạ tên miền nội bộ (service discovery)	bind9, coredns
7	Monitoring Server	Theo dõi CPU, RAM, Network, container	prometheus + node-exporter
8	Logging / Visualization Server	Lưu & hiển thị log hệ thống	grafana, loki, hoặc elastic+kibana
9	Reverse Proxy / Load Balancer	Điều phối truy cập, cân bằng tải	nginx, haproxy, traefik

- Tổng cộng: **9 container**, kết nối chung vào 1 network cloud-net
- Mỗi container nên có Dockerfile riêng (hoặc dùng image có sẵn).

Kiến trúc tổng thể (mô phỏng cloud)



YÊU CẦU DEMO & KIỂM THỬ CHO MỖI SERVER

Loại	Mục tiêu kiểm thử	Cách demo
1. Web Server	Hiển thị index.html, /blog/	Truy cập http://localhost:8080
2. App Server	Trả JSON hoặc API /hello	curl http://localhost:8081/hello
3. Database	Có thể truy vấn dữ liệu	docker exec -it db mysql -e "SHOW DATABASES;"
4. Auth Server	Tạo user, login thành công	Mở giao diện Keycloak/Flask Auth
5. Storage Server	Upload / tải file thành công	Truy cập MinIO Console
6. DNS Server	Có bản ghi nội bộ cho các service	dig @dns-server web-server.cloud.local
7. Monitoring	Prometheus hiển thị metric Node Exporter	Targets: UP
8. Logging / Grafana	Dashboard hiển thị dữ liệu từ Prometheus hoặc Loki	Grafana → http://localhost:3000
9. Reverse Proxy	Cân bằng tải hoặc route domain nội bộ	nginx.conf có upstream ok

YÊU CẦU KỸ THUẬT CHUNG

1. Tất cả container dùng chung mạng nội bộ **cloud-net**
2. Sử dụng **docker-compose.yml** để khởi động toàn hệ thống.
3. Mỗi service có:
 - o Tên container riêng
 - o Exposed port hợp lý (80xx)
 - o Volume (nếu cần lưu dữ liệu)
 - o restart: unless-stopped
4. Push ít nhất 1 image tùy chỉnh (VD: myminicloud-web) lên Docker Hub.
5. Deploy toàn bộ trên **AWS EC2** (Ubuntu), chạy được qua public IP (khuyến khích).

Ví dụ cấu trúc thư mục dự án

```
hotenSVminicloud — root@5e40c90ae82b: / — zsh — 199x54

QAnh@QuangAnh hotenSVminicloud % pwd
/Users/QAnh/LapTrinh/cloud-demo/hotenSVminicloud
QAnh@QuangAnh hotenSVminicloud % tree
.
├── app
│   ├── Dockerfile
│   └── app.py
├── auth
├── db
│   └── init
│       └── 001_init.sql
├── dns
│   ├── db.cloud.local
│   ├── named.conf.local
│   └── named.conf.options
├── docker-compose.ym yes
├── docker-compose.yml
├── grafana
├── monitor
│   └── prometheus.yml
├── proxy
│   ├── empty
│   └── nginx.conf
├── storage
│   └── data
├── web
│   ├── Dockerfile
│   ├── conf.default
│   └── html
│       ├── blog
│       │   └── index.html
│       └── index.html
└── 15 directories, 14 files
QAnh@QuangAnh hotenSVminicloud %
```

BÁO CÁO CUỐI KỲ PHẢI CÓ

Phần	Nội dung
Chương 1. Giới thiệu & mục tiêu	Giải thích hệ thống 9 server, mô hình cloud mô phỏng
Chương 2. Kiến trúc & sơ đồ	Sơ đồ network, flow request giữa server
Chương 3. Cấu hình & Dockerfile	Trình bày các Dockerfile, compose, config
Chương 4. Demo & kiểm thử	Ảnh minh họa mỗi server khi chạy
Chương 5. Đánh giá & phân tích	Nêu ưu điểm – khó khăn – hướng mở rộng
Phụ lục	Link Docker Hub, link video demo, log kết quả

HƯỚNG DẪN CÁC BƯỚC XÂY DỰNG DỰ ÁN

Sử dụng bash/cmd/powershell để thực hiện tạo dự án

0) Khởi tạo dự án

Tạo khung thư mục

```
mkdir -p hotenSVminiclouddemo/{web-frontend-server,application-backend-server,relational-  
database-server,authentication-identity-server,object-storage-server,internal-dns-  
server,monitoring-prometheus-server,monitoring-grafana-dashboard-server,api-gateway-proxy-  
server}  
cd hotenSVminiclouddemo
```

Cổng dùng trong lab

Web 8080, App 8085 (listen 8081 nội bộ), DB 3306, Auth 8081, MinIO 9000/9001, DNS 1053/udp, Prometheus 9090, NodeExporter 9100, Grafana 3000, Proxy 80.

1) Web Frontend —web/ web-frontend-server (Nginx tĩnh)

Mục đích: mô phỏng **static website hosting**; kiểm tra reverse proxy & routing.

```
mkdir -p web-frontend-server/html/blog
```

Trang Home

```
cat > web-frontend-server/html/index.html <<'EOF'  
<h1>MyMiniCloud – Home</h1><a href="/blog/">Blog</a>  
EOF
```

Trang Blog

```
cat > web-frontend-server/html/blog/index.html <<'EOF'  
<h1>MyMiniCloud – Blog</h1><a href="/">Back</a>  
EOF
```

Server block Nginx (chú ý alias có dấu / ở cuối)

```
cat > web-frontend-server/conf.default <<'EOF'  
server {
```

```
    listen 80;  
    server_name _;
```

```
    root /usr/share/nginx/html;  
    index index.html;
```

```
    location / { try_files $uri $uri/ =404; }
```

```
location ^~ /blog/ {
    alias /usr/share/nginx/html/blog/;
    index index.html;
    autoindex off;
}
}
EOF
```

Dockerfile

```
cat > web-frontend-server/Dockerfile <<'EOF'
FROM nginx:stable
RUN rm -f /etc/nginx/conf.d/default.conf
COPY conf.default /etc/nginx/conf.d/default.conf
COPY html/ /usr/share/nginx/html/
RUN chmod -R 755 /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
EOF
```

2) Application Backend — app/application-backend-server (Flask API)

Mục đích: mô phỏng **microservice/API**; có /hello và /secure (OIDC).

Ứng dụng Flask

```
cat > application-backend-server/app.py <<'PY'
from flask import Flask, jsonify, request
import time, requests, os
from jose import jwt

ISSUER = os.getenv("OIDC_ISSUER", "http://authentication-identity-
server:8080/realms/master")
AUDIENCE = os.getenv("OIDC_AUDIENCE", "myapp")
JWKS_URL = f"{ISSUER}/protocol/openid-connect/certs"

_JWKS = None; _TS = 0
def get_jwks():
    global _JWKS, _TS
    now = time.time()
    if not _JWKS or now - _TS > 600:
        _JWKS = requests.get(JWKS_URL, timeout=5).json()
        _TS = now
    return _JWKS

app = Flask(__name__)
```

```

@app.get("/hello")
def hello(): return jsonify(message="Hello from App Server!")

@app.get("/secure")
def secure():
    auth = request.headers.get("Authorization", "")
    if not auth.startswith("Bearer "):
        return jsonify(error="Missing Bearer token"), 401
    token = auth.split(" ", 1)[1]
    try:
        payload = jwt.decode(token, get_jwks(), algorithms=["RS256"], audience=AUDIENCE,
            issuer=ISSUER)
        return jsonify(message="Secure resource OK",
            preferred_username=payload.get("preferred_username"))
    except Exception as e:
        return jsonify(error=str(e)), 401

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8081)
PY

```

Dockerfile

```

cat > application-backend-server/Dockerfile <<'EOF'
FROM python:3.11-alpine
WORKDIR /app
COPY app.py .
RUN pip install --no-cache-dir flask requests python-jose
EXPOSE 8081
CMD ["python", "app.py"]
EOF

```

3) Relational Database — relational-database-server (MariaDB)

Mục đích: mô phỏng **RDS**; tự khởi tạo schema/data lần đầu.

```

mkdir -p relational-database-server/init

cat > relational-database-server/init/001_init.sql <<'SQL'
CREATE DATABASE IF NOT EXISTS minicloud;
USE minicloud;
CREATE TABLE IF NOT EXISTS notes(
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO notes(title) VALUES ('Hello from MariaDB!');

```

SQL

4) Authentication & Identity — authentication-identity-server (Keycloak)

Mục đích: mô phỏng **IdP (OIDC)** phát hành token cho /secure.
Dùng image chính thức, chế độ start-dev, tạo admin bootstrap.

(Không cần file cấu hình riêng — cấu hình sẽ đặt trong docker-compose.yml.)

5) Object Storage — object-storage-server (MinIO)

Mục đích: mô phỏng **S3** (bucket/object). Dữ liệu bên trong ./object-storage-server/data.

```
mkdir -p object-storage-server/data
```

6) Internal DNS — internal-dns-server (Bind9)

Mục đích: mô phỏng **private DNS zone** cloud.local để lab mạng.

```
cat > internal-dns-server/named.conf.options <<'EOF'
options { directory "/var/cache/bind"; allow-query { any; }; recursion yes; };
EOF
```

```
cat > internal-dns-server/named.conf.local <<'EOF'
zone "cloud.local" IN { type master; file "/etc/bind/db.cloud.local"; };
EOF
```

```
cat > internal-dns-server/db.cloud.local <<'EOF'
$TTL 1H
@ IN SOA ns.cloud.local. admin.cloud.local. ( 1 1H 15M 1W 1H )
  IN NS  ns.cloud.local.
ns IN A   127.0.0.1
web-frontend-server IN A 10.10.10.10
EOF
```

7) Monitoring (Prometheus) — monitoring-prometheus-server

Mục đích: thu thập & truy vấn **metric** (sẽ scrape Node Exporter của host/container).

```
mkdir -p monitoring-prometheus-server
```

```
cat > monitoring-prometheus-server/prometheus.yml <<'EOF'
global: { scrape_interval: 15s }
scrape_configs:
  - job_name: 'node'
```

```
static_configs: [ { targets: ['monitoring-node-exporter-server:9100'] } ]
EOF
```

Ghi chú: Node Exporter chạy bằng image chính thức, **không cần thư mục cấu hình**, nên mình không tạo thư mục riêng để giữ đúng “9 server có thư mục”.

8) Grafana Dashboard — monitoring-grafana-dashboard-server

Mục đích: vẽ dashboard từ dữ liệu Prometheus.

(Không bắt buộc file cấu hình ban đầu; cấu hình datasource thực hiện trong UI.)

9) API Gateway / Reverse Proxy — api-gateway-proxy-server (Nginx)

Mục đích: điểm vào duy nhất, định tuyến theo path, strip /api/.

```
mkdir -p api-gateway-proxy-server
```

```
cat > api-gateway-proxy-server/nginx.conf <<'EOF'
events {}
http {
    server {
        listen 80;

        # Web tĩnh
        location / {
            proxy_pass http://web-frontend-server:80;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        # App API (strip /api/)
        location /api/ {
            proxy_pass http://application-backend-server:8081/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        # Keycloak
        location /auth/ {
            proxy_pass http://authentication-identity-server:8080/;
            proxy_set_header Host $host;
```



```

    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
}
EOF

```

10) Docker Compose (chạy cả cụm “mini-cloud”)

Mục đích: mô tả **toàn bộ 9 server + node exporter** (không có thư mục riêng).

docker-compose.yml (đặt ở thư mục hotenSVminiclouddemo)

```
networks: { cloud-net: { name: cloud-net } }
```

services:

web-frontend-server:

build: ./web-frontend-server

image: hotensv/web:dev

container_name: web-frontend-server

networks: [cloud-net]

ports: ["8080:80"]

application-backend-server:

build: ./application-backend-server

image: hotensv/app:dev

container_name: application-backend-server

networks: [cloud-net]

ports: ["8085:8081"]

environment:

OIDC_ISSUER: "http://authentication-identity-server:8080/realms/master"

OIDC_AUDIENCE: "myapp"

relational-database-server:

image: mariadb:11

container_name: relational-database-server

environment:

MARIADB_ROOT_PASSWORD: root

MARIADB_DATABASE: minicloud

volumes:

- ./relational-database-server/init:/docker-entrypoint-initdb.d:ro

ports: ["3306:3306"]

networks: [cloud-net]

authentication-identity-server:

image: quay.io/keycloak/keycloak:latest

container_name: authentication-identity-server
command: ["start-dev"]
environment:
 KC_BOOTSTRAP_ADMIN_USERNAME: admin
 KC_BOOTSTRAP_ADMIN_PASSWORD: admin
 KC_HOSTNAME: "localhost"
 KC_HOSTNAME_STRICT: "false"
 KC_HTTP_ENABLED: "true"
ports: ["8081:8080"]
restart: unless-stopped
networks: [cloud-net]

object-storage-server:
image: minio/minio
container_name: object-storage-server
command: ["server", "/data", "--console-address", ":9001"]
environment:
 MINIO_ROOT_USER: minioadmin
 MINIO_ROOT_PASSWORD: minioadmin
volumes:
 - *./object-storage-server/data:/data*
ports: ["9000:9000", "9001:9001"]
networks: [cloud-net]

internal-dns-server:
image: internetsystemsconsortium/bind9:9.18
container_name: internal-dns-server
ports: ["1053:53/udp"]
volumes:
 - *./internal-dns-server/named.conf.options:/etc/bind/named.conf.options:ro*
 - *./internal-dns-server/named.conf.local:/etc/bind/named.conf.local:ro*
 - *./internal-dns-server/db.cloud.local:/etc/bind/db.cloud.local:ro*
networks: [cloud-net]

monitoring-node-exporter-server:
image: prom/node-exporter:latest
container_name: monitoring-node-exporter-server
ports: ["9100:9100"]
networks: [cloud-net]

monitoring-prometheus-server:
image: prom/prometheus:latest
container_name: monitoring-prometheus-server
ports: ["9090:9090"]
volumes:
 - *./monitoring-prometheus-server/prometheus.yml:/etc/prometheus/prometheus.yml:ro*

networks: [cloud-net]

monitoring-grafana-dashboard-server:

image: grafana/grafana:latest

container_name: monitoring-grafana-dashboard-server

ports: ["3000:3000"]

networks: [cloud-net]

api-gateway-proxy-server:

image: nginx:stable

container_name: api-gateway-proxy-server

depends_on:

- web-frontend-server

- application-backend-server

- authentication-identity-server

ports: ["80:80"]

volumes:

- ./api-gateway-proxy-server/nginx.conf:/etc/nginx/nginx.conf:ro

networks: [cloud-net]

restart: unless-stopped

11) Khởi động & Kiểm thử

Chạy các lệnh sau:

`docker compose build --no-cache`

`docker compose up -d`

`docker compose ps`

Kết quả trả về

```
hotenSVminicloud --zsh -- 229x63

QAnhQQuangAnh hotenSVminicloud % docker compose build --no-cache
docker compose up -d
docker compose ps

[+] Building 5.9s (22/22) FINISHED
=> [internal] load local bake definitions
=> reading from stdin 1.184s
=> [app-server internal] load build definition from Dockerfile
=> transferring dockerfile: 184B
=> [web internal] load build definition from Dockerfile
=> transferring dockerfile: 263B
=> [app-server internal] load metadata for docker.io/library/python:3.11-alpine
=> [web internal] load metadata for docker.io/library/nginx:stable
=> [web internal] load .dockerignore
=> transferring context: 2B
=> CACHED [web 5/5] FROM docker.io/library/nginx:stable@sha256:3b4819335870fb6445987c8d672cf18dec2cbc43b3575581eb66449c8173cd4f
=> resolve docker.io/library/nginx:stable@sha256:3b4819335870fb6445987c8d672cf18dec2cbc43b3575581eb66449c8173cd4f
=> [web internal] load build context
=> transferring context: 168B
=> [web 2/5] RUN rm -f /etc/nginx/conf.d/default.conf
=> [web 3/5] COPY conf.default /etc/nginx/conf.d/default.conf
=> [web 4/5] COPY html/ /usr/share/nginx/html/
=> [web 5/5] RUN chmod -R 755 /usr/share/nginx/html
=> [web] exporting to image
=> exporting layers
=> exporting manifest sha256:34cf1865358863a961c9b91148f864d9e9e8a3675d973bbdda53caa32c
=> exporting config sha256:464cad5f2e295b319ea6388f61167dfc87787cfdab8f9281e4e15cc84772e7
=> exporting attestation manifest sha256:a48d1a2bee789334502c9f886d4ed4abc6aa7fbcf678ba1e6d288979d3723228
=> exporting manifest list sha256:c1fc78ddc8a4eb6c189c4b32768eb6178ee6ba18fc79fa2848514de4e93
=> naming to docker.io/hotensv/web:dev
=> unpacking to docker.io/hotensv/web:dev
=> [web] resolving provenance for metadata file
=> [app-server internal] load .dockerignore
=> transferring context: 2B
=> [app-server 3/4] FROM docker.io/library/python:3.11-alpine@sha256:618ede222c1fa9675c69c99429f8d2c1b4e243f1982246da9e548e5880eeaa
=> resolve docker.io/library/python:3.11-alpine@sha256:618ede222c1fa9675c69c99429f8d2c1b4e243f1982246da9e548e5880eeaa
=> [app-server internal] load build context
=> transferring context: 28B
=> CACHED [app-server 2/4] WORKDIR /app
=> [app-server 3/4] COPY app.py .
=> [app-server 4/4] RUN pip install --no-cache-dir flask requests python-jose
=> [app-server] exporting to image
=> exporting layers
=> exporting manifest sha256:3764a135eaad8bc78299817c9fb85796137aa27451faf5d12037a99c84a9f1d
=> exporting config sha256:797c4353cd638a298e81b734e78dc81c63dc2948acc2392858cc794318f2a8
=> exporting attestation manifest sha256:79714aa4a6b71129e8127d233f8171d2c81489513c7dbb117f3980bd57127
=> exporting manifest list sha256:4bdaefac2374d6f673f8a3918f52b3d39a761c8753721a9e7187ae8777e7a6b
=> naming to docker.io/hotensv/app:dev
=> unpacking to docker.io/hotensv/app:dev
=> [app-server] resolving provenance for metadata file
[+] Building 2/2
  hotensv/web:dev Built
  hotensv/app:dev Built
[+] Running 18/18
  Container hotensvminicloud-keycloak-1 Running
  Container prometheus Running
  Container db Running
  Container mode-exporter Running
  Container web Started
  Container minio Running
  Container grafana Running
  Container dns-server Running
```

1 Web Frontend Server (Nginx static site)

Mục đích: Kiểm tra khả năng phục vụ trang web tĩnh.

Yêu cầu:

Truy cập trang chủ và trang blog, kiểm tra mã phản hồi HTTP.

Lệnh kiểm thử:

```
curl -I http://localhost:8080/
curl -I http://localhost:8080/blog/
```

Kỳ vọng:

- Trả về HTTP/1.1 200 OK.
- Trang Home hiển thị “MyMiniCloud – Home”.
- Trang Blog hiển thị “MyMiniCloud – Blog”.

VD:

- Chạy bằng lệnh trong bash/cmd/powershell

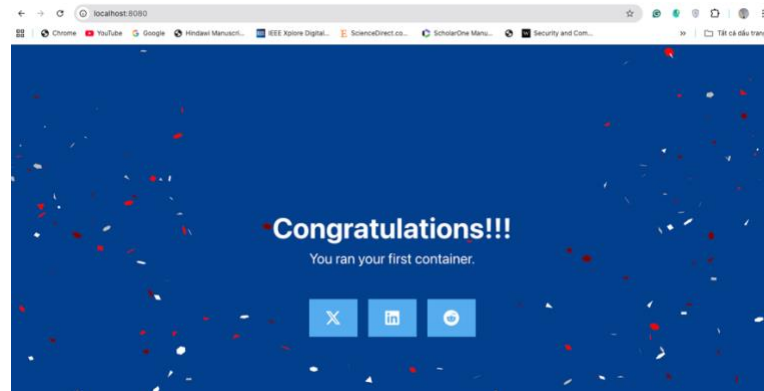
```

[QAnh@QuangAnh hotenSVminicloud % curl -I http://localhost:8080/
HTTP/1.1 200 OK
Server: nginx/1.28.0
Date: Sat, 18 Oct 2025 14:06:45 GMT
Content-Type: text/html
Content-Length: 55
Last-Modified: Sat, 18 Oct 2025 02:31:47 GMT
Connection: keep-alive
ETag: "68f2fc13-37"
Accept-Ranges: bytes

[QAnh@QuangAnh hotenSVminicloud %

```

- Chạy link <http://localhost:8080/>



- Chạy bằng lệnh trong bash/cmd/powershell

```

[QAnh@QuangAnh hotenSVminicloud % curl -I http://localhost:8080/blog/
HTTP/1.1 200 OK
Server: nginx/1.28.0
Date: Sat, 18 Oct 2025 14:11:12 GMT
Content-Type: text/html
Content-Length: 50
Last-Modified: Sat, 18 Oct 2025 02:31:47 GMT
Connection: keep-alive
ETag: "68f2fc13-32"
Accept-Ranges: bytes

[QAnh@QuangAnh hotenSVminicloud %

```

- Chạy link <http://localhost:8080/>



2 Application Backend Server (Flask API)

Mục đích: Kiểm tra API backend hoạt động và trả JSON đúng định dạng.
Yêu cầu:

Gọi API /hello trực tiếp và qua proxy.

Lệnh kiểm thử:

`curl http://localhost:8085/hello`

curl http://localhost/api/hello

Kỳ vọng:

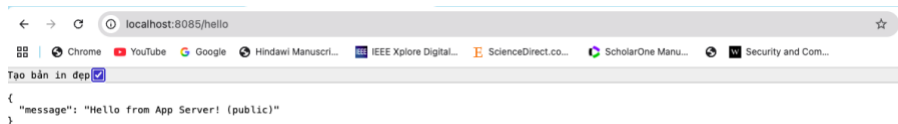
- Trả về JSON: {"message":"Hello from App Server!"}
- Hai lệnh đều hoạt động giống nhau (proxy hoạt động đúng).

Ví dụ

- Chạy bằng lệnh trong bash/cmd/powershell

```
QAnh@QuangAnh hotenSVminicloud % curl http://localhost:8085/hello
{"message":"Hello from App Server! (public)"}
```

- Chạy link <http://localhost:8080/>



3 Relational Database Server (MariaDB)

Mục đích: Kiểm tra dữ liệu khởi tạo tự động trong container DB.

Yêu cầu:

Truy cập database minicloud và đọc bảng notes.

Lệnh kiểm thử:

```
docker run -it --rm --network cloud-net mysql:8 \
sh -lc 'mysql -h db -uroot -proot -e "USE minicloud; SHOW TABLES; SELECT * FROM notes;"'
```

Kỳ vọng:

- Bảng notes tồn tại.
- Có ít nhất 1 dòng dữ liệu Hello from MariaDB!.

```
QAnh@QuangAnh hotenSVminicloud % docker run -it --rm --network cloud-net mysql:8 \
sh -lc 'mysql -h db -uroot -proot -e "USE minicloud; SHOW TABLES; SELECT * FROM notes;"'

mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| Tables_in_minicloud |
+-----+
| notes                |
+-----+
+-----+-----+-----+
| id | title                | created_at |
+-----+-----+-----+
| 1  | Hello from MariaDB! | 2025-10-18 12:56:13 |
+-----+-----+-----+
QAnh@QuangAnh hotenSVminicloud % █
```

4 Authentication Identity Server (Keycloak)

Mục đích: Kiểm tra dịch vụ đăng nhập OIDC hoạt động.
Yêu cầu:

Đăng nhập bằng tài khoản quản trị mặc định.

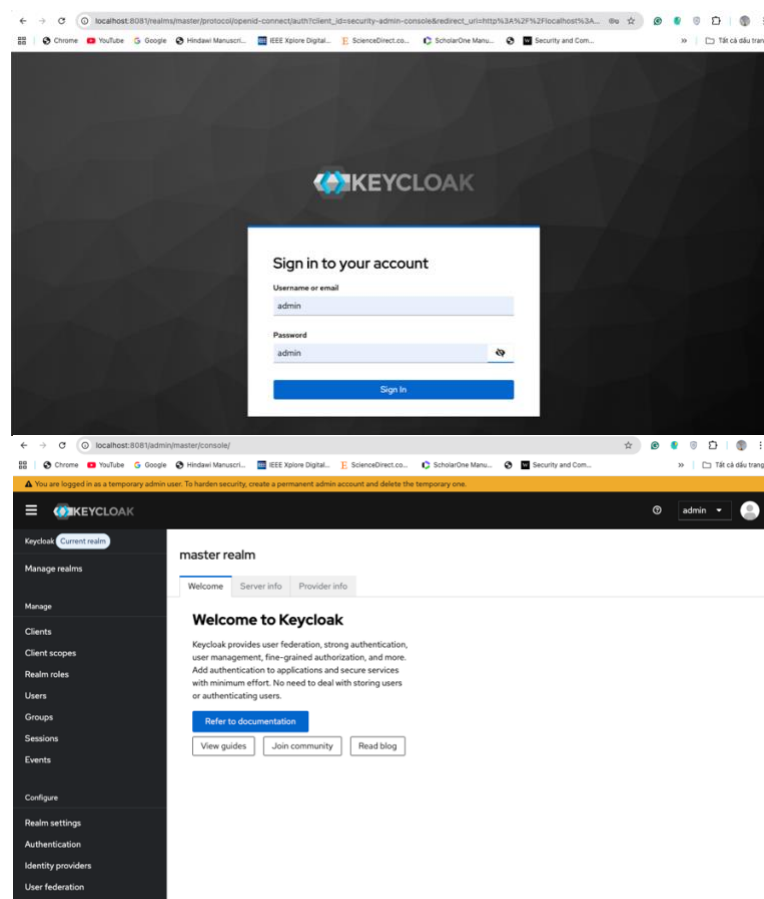
Cách kiểm thử:

- Mở trình duyệt: <http://localhost:8081>
- Đăng nhập bằng:
- Username: admin
- Password: admin

Kỳ vọng:

- Giao diện quản trị hiển thị trang Dashboard.
- Có thể tạo mới 1 user test (sv01).

Ví dụ:



5 Object Storage Server (MinIO)

Mục đích: Kiểm tra lưu trữ đối tượng tương tự Amazon S3.
Yêu cầu:

Truy cập giao diện quản lý, tạo bucket và upload file.

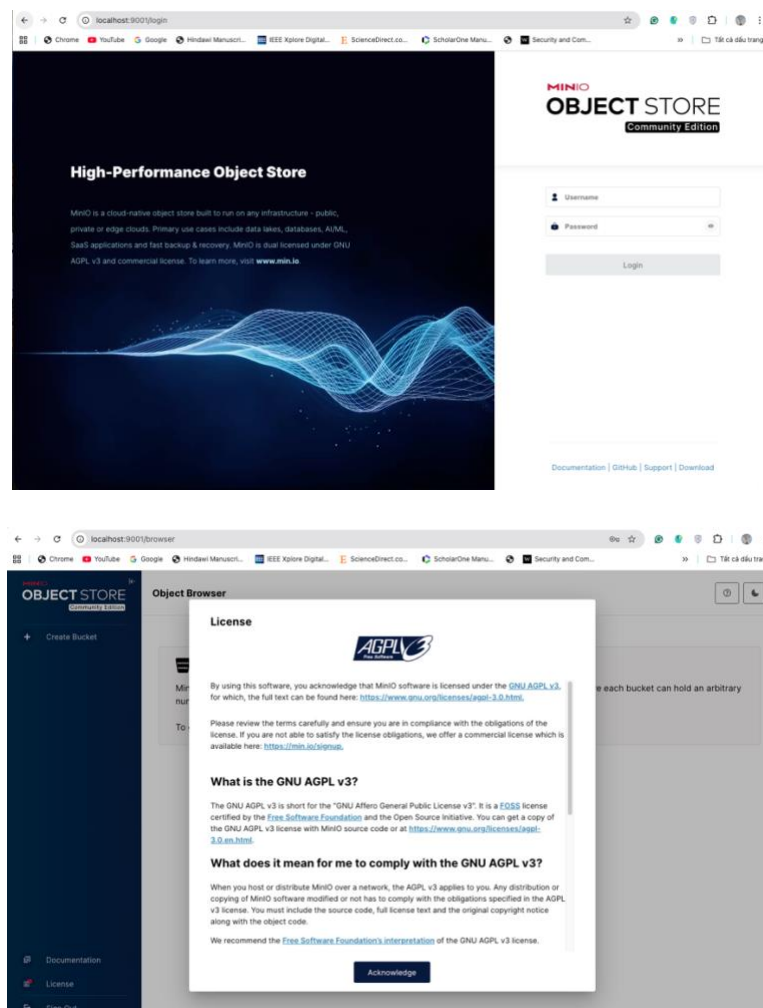
Cách kiểm thử:

- Mở: <http://localhost:9001>
- Đăng nhập: minioadmin / minioadmin
- Tạo bucket tên demo
- Upload file index.html từ thư mục web.

Kỳ vọng:

- File được hiển thị trong bucket demo.

Ví dụ:



6 Internal DNS Server (Bind9)

Mục đích: Kiểm tra phân giải tên miền nội bộ.

Yêu cầu:

Truy vấn bản ghi web-frontend-server.cloud.local.

Lệnh kiểm thử:

dig @127.0.0.1 -p 1053 web-frontend-server.cloud.local +short

Kỳ vọng:

- Trả về IP 10.10.10.10 (theo file cấu hình DNS).
- Nếu trống → kiểm tra db.cloud.local và restart container.

7 Monitoring Node Exporter + Prometheus

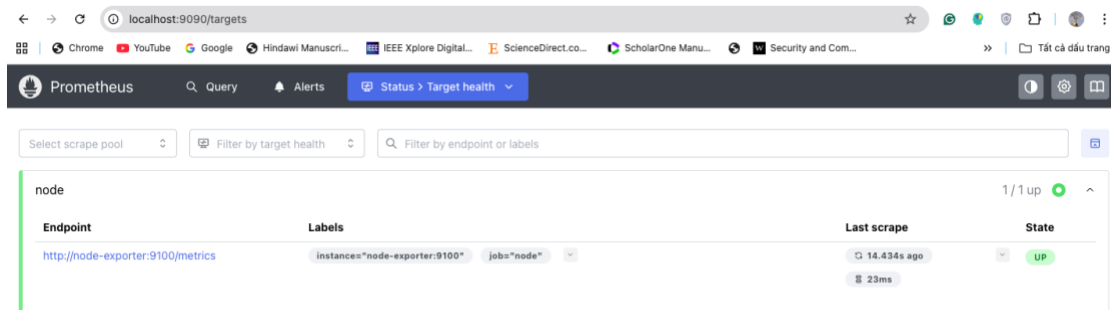
Mục đích: Xác minh hệ thống giám sát thu thập metric hoạt động.

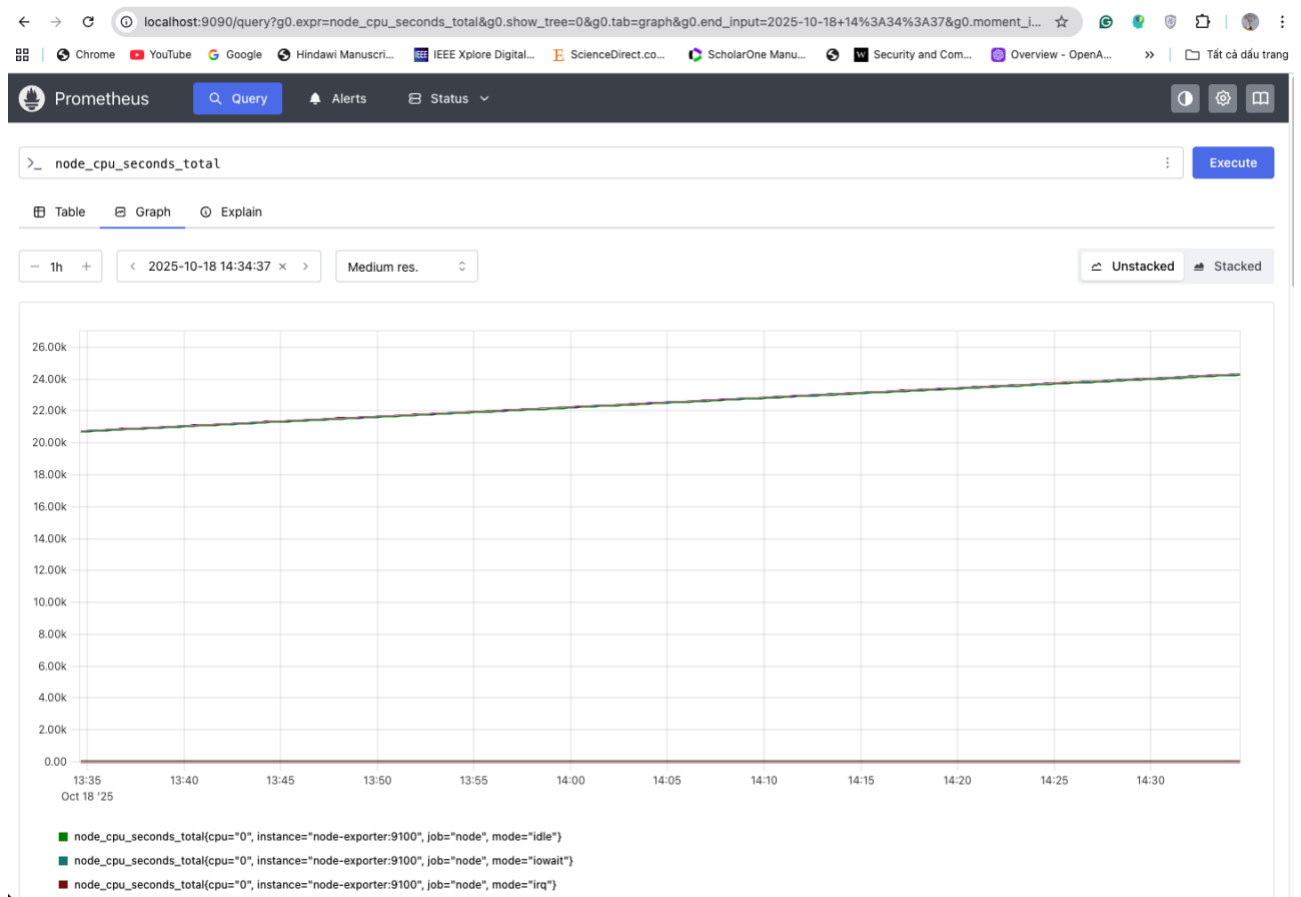
Yêu cầu:

Mở Prometheus, xem Target node-exporter.

Cách kiểm thử:

- Truy cập: <http://localhost:9090>
- Vào: **Status** → **Targets**
- Kiểm tra: monitoring-node-exporter-server:9100 phải **UP**
- Trong tab “Graph”, thử truy vấn:
- node_cpu_seconds_total





8 Monitoring Grafana Dashboard

Mục đích: Kiểm tra khả năng hiển thị biểu đồ giám sát.

Yêu cầu:

Thêm nguồn dữ liệu Prometheus và import dashboard Node Exporter.

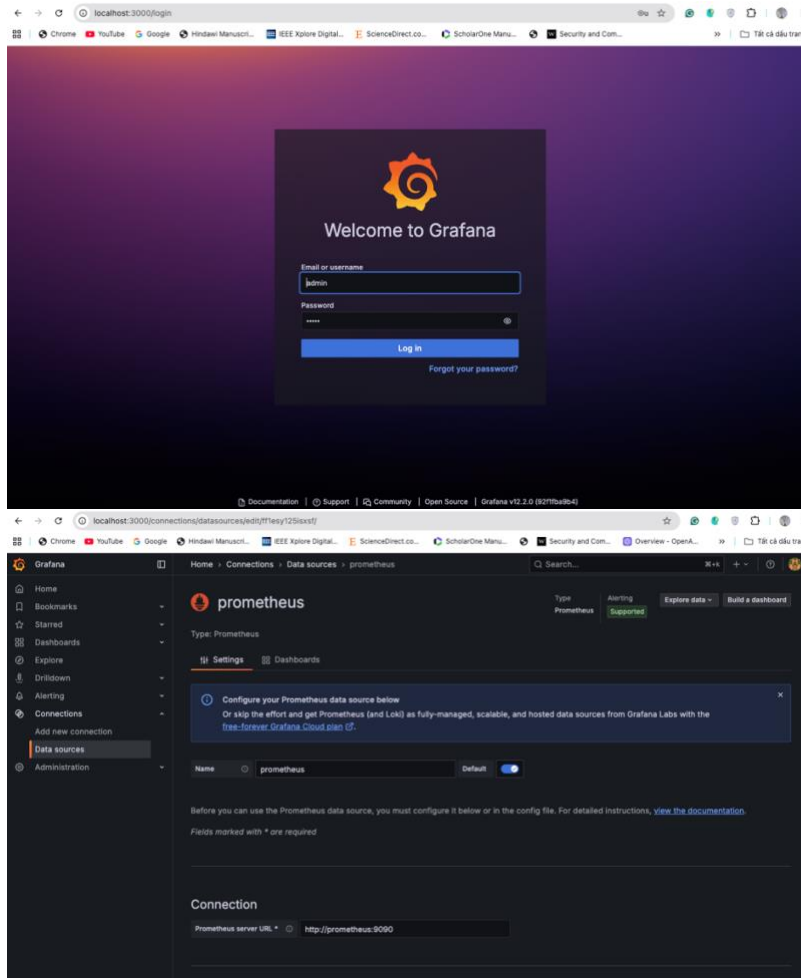
Cách kiểm thử:

- Truy cập: <http://localhost:3000>
- Đăng nhập: admin / admin
- Add datasource → Prometheus → URL: <http://prometheus:9090>
- Import dashboard “Node Exporter Full”.

Kỳ vọng:

- Dashboard hiển thị CPU/RAM/Network metrics.

Ví dụ:



9 API Gateway Proxy Server (Nginx Reverse Proxy)

Mục đích: Kiểm tra routing hợp nhất qua một cổng duy nhất.

Yêu cầu:

Thực hiện 3 truy cập: web, app, auth qua proxy.

Lệnh kiểm thử:

```
curl -I http://localhost/  
curl -s http://localhost/api/hello  
curl -I http://localhost/auth/
```

Kỳ vọng:

- / → trả 200 OK (web).
- /api/hello → JSON từ backend.
- /auth/ → redirect 302 đến Keycloak login page.

```
hotenSVminicloud — root@5e40c90ae82b: / — -zsh — 199x54

[QAnh@QuangAnh hotenSVminicloud % curl -I http://localhost/
HTTP/1.1 200 OK
Server: nginx/1.28.0
Date: Sat, 18 Oct 2025 14:55:08 GMT
Content-Type: text/html
Content-Length: 55
Connection: keep-alive
Last-Modified: Sat, 18 Oct 2025 02:31:47 GMT
ETag: "68f2fc13-37"
Accept-Ranges: bytes

[QAnh@QuangAnh hotenSVminicloud % curl -s http://localhost/api/hello
{"message":"Hello from App Server! (public)"}
[QAnh@QuangAnh hotenSVminicloud % curl -I http://localhost/auth/
HTTP/1.1 404 Not Found
Server: nginx/1.28.0
Date: Sat, 18 Oct 2025 14:55:33 GMT
Content-Type: application/json
Connection: keep-alive

QAnh@QuangAnh hotenSVminicloud % █
```

9 Kiểm trạng thông mạng bằng cách Ping các server khác trong mạng và trả về kết quả thông mạng tất cả

Ví dụ: dùng bash/cmd/powershell

```
ping -c 3 web
ping -c 3 app-server
ping -c 3 db
ping -c 3 keycloak
ping -c 3 minio
ping -c 3 prometheus
ping -c 3 grafana
ping -c 3 dns-server
```

```

hotenSVminicloud — root@5e40c90ae82b: / — docker exec -it hotensvminicloud-proxy-1 sh — 199x54

# ping -c 3 web
ping -c 3 app-server
ping -c 3 db
ping -c 3 minio
ping -c 3 grafana
ping -c 3 prometheus
ping -c 3 node-exporter
ping -c 3 dns-server
ping -c 3 hotensvminicloud-keycloak-1
PING web (172.21.0.11) 56(84) bytes of data.
64 bytes from web.cloud-net (172.21.0.11): icmp_seq=1 ttl=64 time=0.318 ms
64 bytes from web.cloud-net (172.21.0.11): icmp_seq=2 ttl=64 time=0.157 ms
64 bytes from web.cloud-net (172.21.0.11): icmp_seq=3 ttl=64 time=0.131 ms

--- web ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 0.131/0.202/0.318/0.082 ms
# PING app-server (172.21.0.5) 56(84) bytes of data.
64 bytes from app-server.cloud-net (172.21.0.5): icmp_seq=1 ttl=64 time=0.229 ms
64 bytes from app-server.cloud-net (172.21.0.5): icmp_seq=2 ttl=64 time=0.162 ms
64 bytes from app-server.cloud-net (172.21.0.5): icmp_seq=3 ttl=64 time=0.123 ms

--- app-server ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2048ms
rtt min/avg/max/mdev = 0.123/0.171/0.229/0.043 ms
# PING db (172.21.0.6) 56(84) bytes of data.
64 bytes from db.cloud-net (172.21.0.6): icmp_seq=1 ttl=64 time=0.232 ms
64 bytes from db.cloud-net (172.21.0.6): icmp_seq=2 ttl=64 time=0.117 ms
64 bytes from db.cloud-net (172.21.0.6): icmp_seq=3 ttl=64 time=0.149 ms

--- db ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2043ms
rtt min/avg/max/mdev = 0.117/0.166/0.232/0.048 ms
# PING minio (172.21.0.10) 56(84) bytes of data.
64 bytes from minio.cloud-net (172.21.0.10): icmp_seq=1 ttl=64 time=0.200 ms
64 bytes from minio.cloud-net (172.21.0.10): icmp_seq=2 ttl=64 time=0.162 ms
64 bytes from minio.cloud-net (172.21.0.10): icmp_seq=3 ttl=64 time=0.156 ms

--- minio ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2041ms
rtt min/avg/max/mdev = 0.156/0.172/0.200/0.019 ms
# PING grafana (172.21.0.3) 56(84) bytes of data.
64 bytes from grafana.cloud-net (172.21.0.3): icmp_seq=1 ttl=64 time=0.281 ms
64 bytes from grafana.cloud-net (172.21.0.3): icmp_seq=2 ttl=64 time=0.143 ms
64 bytes from grafana.cloud-net (172.21.0.3): icmp_seq=3 ttl=64 time=0.156 ms

--- grafana ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2045ms
rtt min/avg/max/mdev = 0.143/0.193/0.281/0.062 ms
# PING prometheus (172.21.0.4) 56(84) bytes of data.
64 bytes from prometheus.cloud-net (172.21.0.4): icmp_seq=1 ttl=64 time=0.211 ms
64 bytes from prometheus.cloud-net (172.21.0.4): icmp_seq=2 ttl=64 time=0.144 ms
64 bytes from prometheus.cloud-net (172.21.0.4): icmp_seq=3 ttl=64 time=0.147 ms

```

👉 **Chú ý:** xong demo đến đây SV sẽ được 5 điểm

👉 **Mở rộng (5 điểm):** Sinh viên tự làm thêm 9 yêu cầu mở rộng (Ứng dụng thực tế) tương ứng với 9 server trong hệ thống hotenSVminicloud. Mỗi yêu cầu buộc sinh viên vừa hiểu kiến trúc, vừa tùy biến / cấu hình thêm, vừa có sản phẩm cụ thể để kiểm chứng hoạt động thật.

1 Web Frontend Server — web-frontend-server

Mục tiêu học tập: Hiểu cách triển khai website tĩnh và quản lý nội dung.

Yêu cầu mở rộng:

Sinh viên **tạo một blog cá nhân** của riêng mình trong thư mục /blog/ với ít nhất **3 bài viết HTML** theo chủ đề tự chọn (công nghệ, du lịch, học tập, v.v.).

Yêu cầu kỹ thuật:

- Thêm các file: blog1.html, blog2.html, blog3.html.
- Mỗi bài có ảnh minh họa và liên kết quay lại trang chính.
- Cập nhật lại index.html để hiển thị danh sách bài viết (có hyperlink).

Mục tiêu kiến thức:

Hiểu khái niệm **web hosting**, **cấu trúc thư mục web**, và cách **Nginx** phục vụ nội dung tĩnh qua **alias**.

[2] Application Backend Server — application-backend-server

Mục tiêu học tập: Làm quen với **microservice** và **REST API**.

Yêu cầu mở rộng:

Bổ sung một API mới:

/student → trả danh sách sinh viên từ file JSON hoặc database.

Chi tiết:

- Tạo file `students.json` chứa danh sách 5 sinh viên (`id`, `name`, `major`, `gpa`).
- Trong Flask, thêm route:
- `@app.get("/student")`
- `def student():`
- `with open("students.json") as f:`
- `data = json.load(f)`
- `return jsonify(data)`
- Test lại qua proxy: `http://localhost/api/student`

Mục tiêu kiến thức:

Nắm được khái niệm **API endpoint**, **HTTP JSON response**, cách **expose service** nội bộ qua **reverse proxy**.

[3] Relational Database Server — relational-database-server

Mục tiêu học tập: Hiểu về **lưu trữ quan hệ (RDBMS)** và kết nối từ ứng dụng.

Yêu cầu mở rộng:

Sinh viên **tạo cơ sở dữ liệu mới** tên `studentdb` với bảng `students` gồm:

```
id INT PRIMARY KEY AUTO_INCREMENT,  
student_id VARCHAR(10),  
fullname VARCHAR(100),  
dob DATE,  
major VARCHAR(50)
```

và chèn ít nhất **3 bản ghi**.

Mở rộng thêm (tùy chọn):

- Viết truy vấn SELECT, UPDATE, DELETE.
- Kết nối thử từ Flask app để đọc dữ liệu.

Mục tiêu kiến thức:

Thực hành **tạo, đọc, cập nhật, xóa (CRUD)** và hiểu cách container DB lưu dữ liệu qua volume.

4 Authentication Identity Server — authentication-identity-server (Keycloak)

Mục tiêu học tập: Làm quen với **Identity Provider (IdP)** và **Single Sign-On (SSO)**.

Yêu cầu mở rộng:

Sinh viên **tạo 1 Realm mới** tên theo mã sinh viên (ví dụ: realm_sv001) và trong đó:

- Tạo 2 user (sv01, sv02)
- Tạo 1 client tên flask-app (Access Type: public)
- Lấy URL token endpoint và truy cập thử /secure trong app backend.

Mục tiêu kiến thức:

Hiểu mô hình **OIDC**, **realm/client/user**, và **Access Token** trong cloud security.

5 Object Storage Server — object-storage-server (MinIO)

Mục tiêu học tập: Làm quen **lưu trữ phi cấu trúc (object storage)**.

Yêu cầu mở rộng:

Sinh viên tạo 1 bucket tên profile-pics và upload **ảnh đại diện cá nhân**.

Chi tiết:

- Tạo bucket → Upload file avatar.jpg
- Lấy URL public object và kiểm tra truy cập.
- Tạo thêm 1 bucket documents → upload file PDF báo cáo.

Mục tiêu kiến thức:

Hiểu cơ chế **bucket, object, endpoint URL, policy (private/public)** của dịch vụ lưu trữ đám mây.

6 Internal DNS Server — internal-dns-server

Mục tiêu học tập: Hiểu về **phân giải tên miền nội bộ trong cloud**.

Yêu cầu mở rộng:

Sinh viên thêm bản ghi mới trong zone db.cloud.local:

app-backend.cloud.local IN A 10.10.10.20

Sau đó test dig xác minh phân giải được địa chỉ.

Bổ sung:

- Thêm bản ghi minio.cloud.local và keycloak.cloud.local.
- Restart DNS container và kiểm tra lại.

Mục tiêu kiến thức:

Hiểu cơ chế **zone file**, **caching DNS**, và cách các container dùng DNS nội bộ để gọi nhau.

7 **Monitoring Prometheus — monitoring-prometheus-server**

Mục tiêu học tập: Nắm vững nguyên tắc **giám sát metrics & scrape target**.

Yêu cầu mở rộng:

Sinh viên thêm 1 target mới để giám sát **chính web-frontend-server** bằng cách bổ sung đoạn cấu hình:

```
- job_name: 'web'
  static_configs:
    - targets: ['web-frontend-server:80']
```

Thực hiện:

- Sửa prometheus.yml, restart container.
- Mở <http://localhost:9090/targets> để xác nhận target mới **UP**.

Mục tiêu kiến thức:

Hiểu **scrape_configs**, **job_name**, và cách Prometheus thu thập metric qua HTTP endpoint.

8 **Grafana Dashboard — monitoring-grafana-dashboard-server**

Mục tiêu học tập: Hiểu **trực quan hóa dữ liệu và dashboard trong cloud**.

Yêu cầu mở rộng:

Sinh viên **tạo một dashboard cá nhân** tên System Health of <MSSV> gồm ít nhất 3 biểu đồ:

- CPU Usage (%)

- Memory Usage
- Network Traffic

Yêu cầu kỹ thuật:

- Chọn datasource: Prometheus
- Query metric: `node_cpu_seconds_total`, `node_memory_MemAvailable_bytes`, `node_network_receive_bytes_total`
- Lưu dashboard và chụp ảnh gửi minh chứng.

Mục tiêu kiến thức:

Hiểu cách **visualize dữ liệu giám sát** và cấu hình **panel PromQL**.

[9] API Gateway Proxy Server — api-gateway-proxy-server

Mục tiêu học tập: Hiểu về **Reverse Proxy / Routing / Load Balancer**.

Yêu cầu mở rộng:

Sinh viên cấu hình thêm route `/student/` trỏ tới API `/student` của backend.

Thực hiện:

- Mở file `nginx.conf`, thêm:

```
location /student/ {
    proxy_pass http://application-backend-server:8081/student;
}
Restart proxy container:
docker restart api-gateway-proxy-server
Test:
curl http://localhost/student/
```

Kỳ vọng:

Trả về danh sách sinh viên JSON (đọc từ backend).

Mục tiêu kiến thức:

Hiểu cơ chế **proxy_pass**, **route mapping**, và cách API Gateway giúp **tích hợp đa dịch vụ**.

[10] Load Balancer Test — api-gateway-proxy-server mở rộng

Mục tiêu học tập:

Hiểu cơ chế **cân bằng tải (load balancing)** giữa nhiều web server trong môi trường đám mây, sử dụng **Nginx Reverse Proxy** với thuật toán **Round Robin**.

Kỳ vọng:

Cấu hình cân bằng tải cho 2 web server: web-frontend-server1 và web-frontend-server2 bằng thuật toán Round Robin, sau đó kiểm thử luân phiên truy cập.

☑ Tổng kết cấp độ 2 – “Cloud App Integration Practice”

STT	Server	Yêu cầu mở rộng	Mục tiêu kiến thức	Điểm tối đa
1	Web Frontend	Blog cá nhân 3 bài	Web hosting / static site	0,5
2	Backend API	API /student đọc JSON	REST API / proxy routing	0,5
3	Database	Tạo CSDL studentdb + bảng students	CRUD / SQL basics	0,5
4	Keycloak	Realm + client + token test	OIDC / SSO	0,5
5	MinIO	Upload avatar / file PDF	Object storage	0,5
6	DNS	Thêm bản ghi & test dig	Zone & resolution	0,5
7	Prometheus	Add scrape target web	Metrics scraping	0,5
8	Grafana	Dashboard 3 biểu đồ	Data visualization	0,5
9	Proxy	Route /student/ tới backend	Routing / Gateway	0,5
10	load balancing	Nginx Reverse Proxy với thuật toán Round Robin	Kết quả trả về luân phiên giữa 2 trang	0,5
			Tổng cộng	5 điểm

Chú ý, trong báo cáo SV ghi rõ đã làm được bao nhiêu phần trong mục **mở rộng**.